

Module : Apprentissage Artificiel (Machine Learning)

Filière Parcours d'Excellence ISOC

Semestre 6

TP3 : La Régression Linéaire Multiple

Responsable : Brahim AKSASSE
FS Meknès
Département Informatique

Année Universitaire : 2025-2026

1. Introduction

Le but de ce TP de programmation est de considérer le cas d'une variable dépendante $y^{(i)}$ en fonction de plusieurs variables indépendantes $x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)}$ en prenant l'exemple de House Pricing. Le prix d'un appartement $y^{(i)}$ est fonction de plusieurs variables indépendantes $x_j^{(i)}$ (la surface de l'appartement, nombre de chambre,...). Comme en TP2, nous allons les programmer avec Matlab. Dans ce TP, vous étudierez la régression linéaire multivariée à l'aide de l'algorithme Gradient Descent, la méthode de Newton et des équations normales. Vous examinerez également la relation entre la fonction de coût $J(\theta)$ et la convergence de l'algorithme Gradient Descent et le taux d'apprentissage α .

2. Manipulation des données

Téléchargez tp3Data.rar et extrayez les fichiers du fichier rar.

Il s'agit d'un ensemble d'apprentissage des prix des logements à Portland, Oregon, où les sorties $y^{(i)}$ sont les prix et les entrées $(x_1^{(i)}, x_2^{(i)})$, $i = 1 \dots, m$ sont la surface habitable et le nombre de chambres de l'exemple i . Il existe au total $m=47$ exemples que nous allons utiliser pour développer un Modèle de régression multiple.

3. Prétraitement de vos données

Chargez les données des exemples d'entraînement dans votre programme et ajoutez le terme d'interception $x_0^{(i)} = 1$ dans votre matrice de données x . Rappelons que la commande dans Matlab pour ajouter une colonne de uns est

```
x = [ones(m, 1), x];
```

Jetez un œil aux valeurs des entrées $x_1^{(i)}, x_2^{(i)}$ et notez que les espaces des appartements représentent environ 1000 fois le nombre de chambres. Cette différence signifie que le prétraitement des entrées augmentera considérablement l'efficacité de la descente de gradient (normalisation et standardisation).

Dans votre programme, mettez à l'échelle les deux types d'entrées en fonction de leurs écarts types et centrer bien leurs moyennes sur zéro. Dans Matlab, cela peut être exécuté avec

```
sigma = std(x);  
mu = mean(x);  
x(:,2) = (x(:,2) - mu(2)) ./ sigma(2);  
x(:,3) = (x(:,3) - mu(3)) ./ sigma(3);
```

4. La régression linéaire multiple et l'algorithme Gradient Descent

Maintenant, nous allons implémenter la régression linéaire multiple ($n=2$) pour ce problème. Rappelons que le modèle de régression linéaire est

$$\hat{y}^{(i)} = h_{\theta}(x^{(i)}) = \theta^T x^{(i)} = \sum_{j=0}^n \theta_j x_j^{(i)}$$
$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}$$

La règle de mise à jour du vecteur paramètres par Batch Gradient descent est

$$\theta_j = \theta_j - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad j = 0, 1, \dots, n$$

Ou la forme matricielle

$$\theta = \theta - \frac{\alpha}{m} (X^T (h_{\theta}(X) - Y))$$

5. Sélection d'un taux d'apprentissage à l'aide de $J(\theta)$

Il est maintenant temps de sélectionner un taux d'apprentissage α . Le but de cette partie est de choisir un bon taux d'apprentissage dans la gamme de $0,001 \leq \alpha \leq 10$

Pour ce faire, effectuez une sélection initiale, exécutez l'algorithme Gradient Descent et observez la fonction de coût $J(\theta)$, et ajustez le taux d'apprentissage en conséquence. Rappelons que la fonction de coût est définie comme

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

La fonction de coût peut également être écrite sous la forme vectorisée suivante :

$$J(\theta) = \frac{1}{2m} (X\theta - Y)^T (X\theta - Y)$$

Où $X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} \end{bmatrix}, Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$

La version vectorisée est utile et efficace lorsque vous travaillez avec des outils de calcul numérique comme en Algèbre linéaire en Matlab. Si vous êtes familier avec les matrices, vous pouvez vous prouver que les deux formes sont équivalentes.

Vous allez maintenant calculer la fonction coût $J(\theta)$, pour chaque étape actuelle l'algorithme Gradient Descent. Après avoir parcouru de nombreuses étapes, vous verrez comment $J(\theta)$ change à mesure que les itérations avancent.

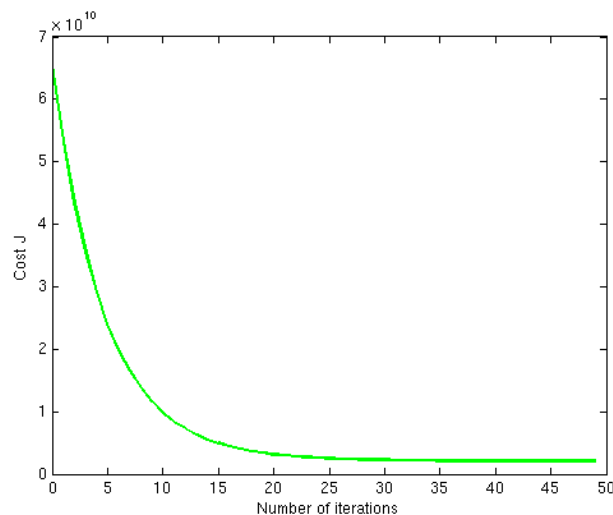
Maintenant, exécutez la descente de gradient pendant environ 50 itérations à votre rythme d'apprentissage initial. À chaque itération, calculez coût $J(\theta)$ et stockez le résultat dans un vecteur J. Après la dernière itération, tracez les valeurs J par rapport au numéro de l'itération. Vous pouvez utiliser le code suivant :

```
theta = zeros(size(x(1,:)))'; % initialize fitting parameters
alpha = %% Your initial learning rate %%
J = zeros(50, 1);

for num_iterations = 1:50
    J(num_iterations) = %% Calculate your cost function here %%
    theta = %% Result of gradient descent update %%
end

% now plot J
% technically, the first J starts at the zero-eth iteration
% but Matlab doesn't have a zero index
figure;
plot(0:49, J(1:50), '-');
xlabel('Number of iterations')
ylabel('Cost J')
```

Si vous avez choisi un taux d'apprentissage dans une bonne fourchette, votre tracé devrait ressembler à la figure ci-dessous.



Si votre graphique semble très différent, surtout si votre valeur de $J(\theta)$ augmente ou même diverge, ajustez votre taux d'apprentissage et réessayez. Nous vous recommandons de tester les alphas à un taux de 3 fois la valeur la plus petite suivante (c'est-à-dire 0,01, 0,03, 0,1, 0,3, etc.). Vous pouvez également ajuster le nombre d'itérations que vous exécutez si cela vous aide à voir la tendance générale de la courbe.

Pour comparer l'impact de différents taux d'apprentissage sur la convergence, il est utile de tracer J pour plusieurs taux d'apprentissage sur le même graphique. Dans Matlab cela peut être fait en effectuant une descente de gradient plusieurs fois avec une commande «hold on» entre les tracés. Concrètement, si vous avez essayé trois valeurs d'alpha différentes (vous devriez probablement essayer plus de valeurs que celle-ci) et stocké les coûts en J1, J2 et J3, vous pouvez utiliser les commandes suivantes pour les tracer sur la même figure :

```
plot(0:49, J1(1:50), 'b-');
hold on;
plot(0:49, J2(1:50), 'r-');
plot(0:49, J3(1:50), 'k-');
```

Observez les changements dans la fonction de coût à mesure que le taux d'apprentissage change. Que se passe-t-il lorsque le taux d'apprentissage est trop faible ? Trop grande ?

En utilisant le meilleur taux d'apprentissage que vous avez trouvé, exécutez la descente de gradient jusqu'à la convergence pour trouver

- Les valeurs finales de θ
- Le prix prédit d'une maison de 1650 pieds carrés et 3 chambres à coucher. N'oubliez pas de mettre à l'échelle vos fonctionnalités lorsque vous faites cette prédiction ! (normalisation)

6. La solution par les équations normales

On peut retrouver la solution pour l'apprentissage du vecteur paramètres en utilisant les équations normales. Posons

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} \end{bmatrix}, Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

$$\hat{\theta} = \frac{1}{m} (X^T X)^{-1} (X^T Y)$$

Comparer les deux estimateurs du vecteur paramètres

7. La solution par la méthode de Newton

La méthode de Newton-Raphson est une méthode itérative mais beaucoup plus rapide que l'algorithme du gradient descent. En quelques itérations, vous allez obtenir la solution pour le vecteur paramètres

$$\theta = \theta - H(\theta)^{-1} \nabla_{\theta}(l(\theta))$$

Comparer les trois estimateurs obtenus pour le vecteur paramètres.